COMMERCIAL HOSPITALITY SPECIFICATION

OZLODGE PLATFORM

Data-Sovereign Commercial Smart-Lock Infrastructure

Author: Truong Viet Phan (Vince Phan)

Organization: eBizco Australia Pty Ltd

Date: August 2026

Status: Technical Product Specification — Version 2.2

OZLODGE is the commercial hospitality tier of the Sovereign Edge smart-lock platform, engineered specifically for hotels, motels, short-stay accommodations, student housing, and mining accommodation. Grounded in the Sovereign Edge trust model, the platform resolves the post-purchase "hold-up" problem through a combination of open local-first runtime control, open-source compliance under GPLv3 copyleft licenses, and hardware credential containment. **This document serves as the formal architectural specification for Version 2.2.**

1. EXECUTIVE SUMMARY

1.1. Introduction & Core Philosophy

1.1.1. The Sovereign Edge Mission

Connected hardware products routinely place consumers in a position transaction cost economics calls *hold-up*: once a customer has made a relationship-specific investment—installing a lock, wiring a hub—the vendor who controls the supporting cloud can unilaterally alter terms, degrade functionality, or withdraw support altogether. This smart-lock specification develops a structural defense against hold-up that is architectural rather than contractual, ensuring that data, credentials, and hardware control remain in the hands of the property owner.

1.2. The Sovereign Edge Paradigm

1.2.1. Admission vs. Runtime Sovereignty

By establishing clear separation between runtime control (the daily operation of the physical locks) and admission control (who is allowed to construct and connect compatible devices), the platform occupies the vacant 'Open-Open' quadrant of the platform-governance matrix. Admission is kept open through permissible MIT protocol licensing, preventing consortium gatekeeping and five-figure compliance assessments. Runtime control is kept open by utilizing end-to-end payload encryption and on-premises local database authorities, ensuring the server acts purely as a content-blind relay.

1.3. Document Scope & Audience

1.3.1. Technical Scope

This technical specification acts as the definitive system architecture manual for the commercial **OZLODGE** tier of smart locks. It details physical local networking topologies, security architectures, cryptographic parameters, operational check-in and booking workflows, and maintains the dated Status Register verifying technical capability claims. It serves as the primary system of record for technical managers, property managers, software engineers, and distribution partners.

2. SYSTEM ARCHITECTURE

2.1. Network Topologies

2.1.1. On-Premises LAN Path

For daily commercial hotel operations, the primary property management system application—the **MAOI PMS App**—communicates directly with the local server **OZLODGE** (a.k.a OZKEYSERV, running locally on port :3200) over the shared hotel Wi-Fi LAN. This direct connection ensures high performance and sub-second room status synchronization. All communications between the MAOI tablet and the local server are encrypted and require initial BLE pairing during staff enrollment.

2.1.2. Cloud Remote Path

For off-site and remote management, the MAOI app tunnels to the local server through the secure **NEXUS [CLOUD]** server. If a property does not host its own Virtual Private Server (VPS), the **OZLOCKSERV** (provided as a free public MQTT server by eBizco Australia) acts as a STUN/MQTT conduit to relay messages. Properties are fully entitled to deploy their own self-hosted OZLOCKSERV on private cloud servers, modifying the bridge and app connection URLs accordingly.

2.1.3. Guest Interaction Path

Guest integrations utilize a multipurpose application—the **BANOI App**—acting as a digital key container. Check-in requests require GPS confirmation to verify the guest is physically within 500 meters of the hotel property. Once verified, the 'CheckIn' button appears. Requests route from BANOI → NEXUS (acting as MQTT relay) → OZKEYSERV. The local server processes the PMS booking, auto-generates a secure digital key, and relays it back to BANOI. Simultaneously, OZKEYSERV updates the physical door lock with the digital key and validity duration down-mesh via the Thread/Wi-Fi bridge.

2.2. Component Descriptions

2.2.1. Smart Lock Hardware

Physical door locks are built on Espressif ESP32-C6 N8 microcontrollers operating in Full Thread Device (FTD, rx_on=1) mode for sub-second responsive mesh communication. PIN keypad entry and RFID cards are processed 100% offline against the lock's local database and stored public keys, ensuring immediate access even during catastrophic network outages.

2.2.2. Wi-Fi / Thread Bridge

Mains-powered ESP32-C6 N16 gateway devices acting as local Thread Border Routers and MQTT uplinks. Bridges route end-to-end encrypted payloads down to individual door locks. Typical deployment density ranges from 8 to 16 locks per bridge, depending on physical floor layouts.

2.2.3. OZKEYSERV Local Server

The physical local hardware brain on hotel premises. It runs a local MySQL instance storing room rosters, backup PMS database tables, audit trails, and coordinates with third-party payment and SMS gateways locally.

2.2.4. MAOI PMS App

The core front-desk property management software running on staff tablets. MAOI serves as the ultimate authority for staff operations—allocating room assignments, managing shift rosters, issuing credentials, and reviewing audit trails. It has a local primary database which is backed up to the OZKEYSERV PMS database. A secondary variation of the MAOI app is deployed on public lobby **Self-Check-In Kiosks**.

2.2.5. BANOI App

A multipurpose guest-facing mobile application. BANOI manages personal residential smart locks, performs commercial room bookings, and manages check-ins. In-stay guest BLE unlocks negotiate directly with locks via cryptographic Digital Passports.

2.2.6. NEXUS Cloud Server

The central cloud-hosted identity and registration authority mainly for MAOI and BANOI apps, providing user authentication, profile matching, and optional cloud database backups for hotel property management logs.

2.3. System Architecture Diagram

2.3.1. Topology Visual Layout — Encrypted End-to-End Flow

ERROR: Image 'ozlodge-hospitality-architecture-v2.png' not found.

Figure 1: *Scale-up hospitality network topology. Operations route from the MAOI PMS app and check-in kiosks on the physical hotel LAN directly to the local server OZKEYSERV. Guest booking sync and check-in exchanges route via the BANOI app through the secure NEXUS cloud directory down to the local server. Opaque encrypted command frames are pushed over the Thread mesh via ESP32-C6 bridges, where they are executed offline by local door locks.*

3. TRUST & SECURITY MODEL

3.1. Cryptographic Enforcements

3.1.1. AES-256-GCM Payload Encryption

All commands sent to a physical lock, including PIN codes and digital keys, are encapsulated inside AES-256-GCM encrypted envelopes. The payload contains an incremental counter (counter_floor) preventing replay attacks and is cryptographically bound to the target device ID via Authenticated Additional Data (AAD). The server relays the hex envelope verbatim, maintaining structural content blindness.

3.1.2. X25519 ECDH Key Exchange

During device commissioning, the lock and the owner's app establish a secure, shared pairing secret using X25519 Elliptic Curve Diffie-Hellman (ECDH). The public server never obtains these key secrets, meaning it is mathematically impossible for the cloud vendor, a subpoenaed server, or a malicious employee to decrypt control envelopes or forge commands.

3.1.3. Local Microcontroller Authority

The absolute authority to grant entry is localized on the lock's microcontroller. Credentials (PINs, RFID tokens, and member bonds) are stored in the lock's local flash. Expiry enforcement and digital passports are checked offline, meaning locks operate fully independently of a persistent network connection.

3.2. Data Sovereignty and Minimization

3.2.1. Pure Relay Server Design

The NEXUS and OZLOCKSERV systems act as message relays rather than data hoards. The cloud contains only a device-to-app routing pairing map. It stores no transaction logs, no user names, and no plaintext credentials, minimizing the attack surface and satisfying key provisions of the EU Cyber Resilience Act (CRA) at the architectural layer.

3.2.2. Log Plaintext Transmission Gap

In keeping with technical transparency, a known security gap currently exists (Claim C7 / L8). Currently, lock opening logs (publishLog()) containing the device ID, timestamp, and holder class are transmitted in plaintext over the MQTT log topic. The development team has specified a remediation to seal this log payload within the ozkey-17 secure uplink, which will enforce complete content blindness once implemented.

4. KEY OPERATIONAL WORKFLOWS

4.1. OTA Booking & Lobby Kiosk Check-In

4.1.1. PMS Booking Capture

A guest books a room on a traditional OTA. The hotel's Property Management System processes the booking, and front-desk staff input the reservation directly into the **MAOI PMS App**. MAOI synchronizes the room definitions and duration parameters to the OZKEYSERV local database over the hotel LAN.

4.1.2. Kiosk Enrollment and Key Distribution

Upon arrival, the guest approaches the lobby's physical Self-Check-In Kiosk (running the Secondary MAOI App). The kiosk queries the local on-premises server, matches the reservation, and allows the guest to complete enrollment. The kiosk can print or program a physical RFID key card, or display a secure keypad PIN. This PIN is programmed directly into the lock's MCU via the local Wi-Fi/Thread bridge.

4.2. Mobile BANOI App Geofenced Self-Check-In

4.2.1. OTA to BANOI Sync

If the hotel operator elects mobile check-in, the OZKEYSERV server registers the guest's booking and alerts NEXUS, which triggers an automated guest invitation via SMS or email containing a BANOI App download link. Upon account registration, the hotel booking is synchronized down to the guest's booking calendar page.

4.2.2. Geofenced Key Provisioning

When the guest arrives within a 500-meter radius of the property, the BANOI App displays a 'CheckIn' button via GPS geofencing. The guest taps check-in, sending an encrypted handshake from BANOI → NEXUS (acting as STUN/MQTT relay) → OZKEYSERV. The local server generates a unique time-bound digital passport (X25519 member bond) and returns it to BANOI over a secure TLS channel. Simultaneously, OZKEYSERV pushes the credential down to the physical lock's MCU. At the door, the guest wakes the lock by touching the keypad and performs an instant local BLE unlock.

5. COMPARISON & DIFFERENTIATION

5.1. Comparison with Tuya/TTLock

5.1.1. Data Sovereignty Evaluation

Traditional white-label platforms represent a closed-runtime model where all guest data, door event history, PINs, and credentials are held centrally in cloud systems subject to third-party or foreign jurisdictions. OZLODGE implements a sovereign runtime model where the hotel operator maintains physical custody of all guest transaction data.

Aspect / Feature	Tuya / TTLock (White-Label)	OZLODGE (Sovereign Edge)
Cloud Owner	Third-party (PRC-domiciled Tuya cloud)	Hotel operator (Self-hosted on on-prem or private VPS)
Data Sovereignty	Data is owned/controlled by the vendor	Hotel maintains 100% data ownership
Credential Storage	PINs, RFID, and biometric tokens in plaintext on the cloud	Envelopes are encrypted end-to-end; never stored on cloud
Server Visibility	Can read and log every command, PIN, and door lock status	Opaque envelopes; cloud sees only routing metadata
Transaction Logs	Stored indefinitely on third-party servers	Stored only locally on the hotel's on-premises server
Offline Operation	Severely limited; requires active internet for smart workflows	Fully operational offline via local MCU and BLE bonds
Vendor Lock-In	Complete dependence on proprietary protocol and servers	None; open-source (GPLv3) and open protocol (MIT)

5.2. Technical Moat & Strategic Positioning

5.2.1. Multi-Tier License Architecture

To secure the platform's open-open paradigm, OZLODGE uses a differentiated licensing strategy as an architectural hostage. The protocol specification and SDK are published under the permissive **MIT License**, encouraging adoption and integration. Conversely, the server codebase, bridge firmware, and BANOI mobile app are released under the strong copyleft **GPLv3 License**. GPLv3's anti-tivoization clause legally prevents the manufacturer from closing the platform or signing hardware in a way that blocks legitimate refleshing. This guarantees the hotel has a permanent, technical exit route—allowing them to fork the codebase and self-host without losing lock function or vendor cooperation.

6. SYSTEM IMPLEMENTATION STATUS

6.1. Revision History & Overview

6.1.1. August 2026 Revision Summary

To maintain rigorous compliance and technical accuracy, the technical team has audited and corrected previous specification claims against physical bench evaluations. The status register has been corrected on three key claims: (1) Disclosed that lock logs (publishLog()) currently route in plaintext on MQTT topics (C7/L8), with encryption specified; (2) Corrected the lock sleep state, confirming the lock operates with receiver-on (Full Thread Device) and the 60-600s interval represents the heartbeat frequency; (3) Added a supply-chain disclosure that standard-tier Espressif C6 chips are PRC-domiciled, and verified US-made Silicon Labs EFR32MG24 chips as the compliant alternative for high-security defense tier procurement.

6.2. Status Register

6.2.1. Hospitality Claim Verification Status

ID	Claim Description	Status	Bench Evidence / Artifact
L1	Sealed envelopes; hotel server cannot read credentials in transit	**VERIFIED**	Inherits OZLOCK C1/C2
L2	PIN stored on lock MCU, works offline	**VERIFIED**	DP 21/22 path, bench-verified
L3	Digital passport = X25519 member bond on lock	**VERIFIED**	M3 member ceremony; phone-to-phone verified
L4	Revocation is lock-side (DP 22 / DP 101)	**VERIFIED**	REVOKE_OK + roster confirms removal
L5	Remote unlock sub-second via bridge	**VERIFIED**	Measured ~240-330 ms lock-side relay
L6	Lock sleeps 5s-15min (configurable)	■ **INCORRECT**	Corrected in 1.1: FTD mode, rx_on=1, 60-600s is heartbeat
L7	Multi-year battery at hospitality duty cycle	■■ **UNMEASURED**	FTD mode battery consumption unmeasured
L8	Full audit trail on hotel's own server	**ACHIEVABLE**	Framing corrected; doesn't conflict with E2E blind relay
L9	After-hours QR self-check-in	■ **DESIGN ONLY**	Phase 5 development milestone, not built
L10	PMS roster sync via /pms/rooms	**BUILT**	Roster sync endpoints built; upsert/reconcile validated

6.2.2. Inherited Shared Platform Claims

ID	Claim Description	Status	Bench Evidence / Artifact
C1	Commands are AES-256-GCM sealed end-to-end; server cannot read them	**VERIFIED**	ozkey-06 envelope, byte-verified against envelope.dart
C2	Server relays envelope_hex verbatim, never decrypts	**VERIFIED**	ozkey-13 S1-S9 cutover; buildCredentialFrame() deleted
C3	Monotonic counter prevents replay	**VERIFIED**	counter_floor per bond; verified across reboot (U0 reservation)
C4	Credentials (PIN/RFID) never stored on server in plaintext	**VERIFIED**	ozkey-13 §4; server holds credential metadata only
C5	Lock works offline via BLE and PIN	**VERIFIED**	PIN on MCU; BLE unlock independent of any network
C6	Lock-to-app channel is sealed	**VERIFIED**	ozkey-17 U1; first production use 2026-08-10
C7	Server stores no record of who opened which door	■■ **GAP**	publishLog() currently sends logs in plaintext. Remediation specified
C8	Sleepy End Device status	■ **INCORRECT**	Corrected in 2.1: firmware is Full Thread Device (ftd=1) for sub-second relay
C9	Multi-year battery life in mesh config	■■ **UNMEASURED**	No FTD measurement exists under realistic mesh traffic
C10	Self-hostable relay, open source	**VERIFIED**	Source in-repo; GPLv3/MIT licences verified